

Question 1: State Lifting & Architecture

The `_themeMode` variable lives in `_RunMyAppState`, at the very top of the widget tree. Suppose the toggle control were moved three widgets deeper into a nested child. What specifically would break, and what mechanism (e.g., callbacks, `InheritedWidget`, or a state management package) would you use to fix it? Explain why "lifting state up" was the right call here instead of declaring a second, local `_themeMode` variable inside the child widget.

The theme would not be able to update because it would be unable to see what thememode it is on since thememode is in a child widget. This can be fixed by sending the themeMode variable back up to the widget (aka a callback). This is better than declaring a second local variable because declaring a local variable would mean that you would need to change both variables every time you want to change that theme. This creates a single source of truth.

Question 2: DRY Principle & Theme.of(context)

Every widget in your app reads colors through `Theme.of(context)` instead of hardcoded values like `Colors.grey`. Walk through a concrete scenario where hardcoding colors in five different widgets would cause a maintenance problem later. Then explain, at a technical level, how `Theme.of(context)` locates the nearest `Theme` in the widget tree and why that lookup mechanism enables a single source of truth for styling.

Hardcoding colors would cause a maintenance problem because whenever you would want to change the colors of the widget, you would have to go through every part of the widget to change each color individually. A theme stores the colors in one place, so you can just change the color there.

`Theme.of(context)` starts by looking at its current widget to see if that widget has a theme. If it does not, it goes up one layer (to its parent) and tries to find a widget there. It repeats this until it finds a `Theme` or it reaches the root `Widget`. This creates a single source of truth because at any point, it is only possible to find exactly one theme, so there is no confusion.

Question 3: Scalability Beyond Light/Dark

Imagine your instructor now asks you to support a third mode: "High Contrast." Would the `ThemeMode` enum (which only has `system`, `light`, and `dark`) still work? Identify exactly where in your code you would need to make changes (variable types, conditional logic, widget properties) to support a custom third mode, and explain any trade-offs of the approach you choose.

The `ThemeMode` enum would not work. I think what I would do is mimic the `ThemeMode` enum by creating a new class with the current `ThemeMode` enum properties and the new enums. The trade off is the reliance on a new file that is not native to the Flutter code which increases the application size.

Question 4: Performance & UX Trade-offs of Animation

Compare animating a single `AnimatedContainer` (Part 2) against wrapping the whole screen in an `AnimatedTheme` (Special Feature Section). Discuss at least one performance consideration (e.g., how many widgets must rebuild/repaint) and one user-experience consideration (e.g., visual consistency) for each approach. Which would you choose for a production app with many themed widgets, and why?

One performance consideration is if there are a decent amount of widgets that either need to change their theme independent of each other or do not change their theme. Less widgets that need to change their theme or more widgets that need to change their theme independently of each other makes doing animated containers better than animated themes. One user-experience consideration is how the app will look. Having every theme change at the same time could be a poor experience, so having themes change individually may lead to a better user experience. I would choose individual `Animated Containers` for an app with many themed widgets because it gives me better control over what themes change for individual widgets at what time.